Course Code	**: MCS-208**
Course Title	**: Data Structures and Algorithms**
Assignment Number	**: BCA_NEW(I)-208/Assignment/2025-26**
Maximum Marks	**: 100**
Weightage	**: 25%**
Last Date for Submission	**: 31th October 2025 (for July Session)** **30th April, 2026 (For January 2026 Session)**

There are four questions in this assignment, which carry 80 marks. Each question carries 20 marks. Rest 20 marks are for viva voce. All algorithms should be written nearer to C programming language. You may use illustrations and diagrams to enhance the explanation, if necessary. Please go through the guidelines regarding assignments given in the Programme Guide for the format of presentation.

Q1: For each of the Singly Linked List, Circularly Singly Linked List, Doubly Linked List, Circularly Doubly Linked List, write one application that is exclusively suitable for that list. For example, X may be an application for whose implementation, only Circularly Singly Linked List is suitable and others are not suitable. Justify your answer. (20Marks)

Solution:

1. Singly Linked List (SLL)

Application: Implementing a **Stack** data structure.

Justification:

- A stack requires insertion and deletion operations at only one end (top of the stack).
- Singly linked lists support efficient insertion and deletion at the head with O(1) time complexity.
- There is no need to traverse backward or access previous elements, making an SLL the simplest and most efficient for stack operations.

2. Circularly Singly Linked List (CSLL)

Application: Implementing a **Round Robin Scheduling Algorithm** in operating systems.

Justification:

- Round Robin scheduling needs to cycle through processes in a circular manner without termination, passing the CPU time slice to each process repeatedly.
- CSLL's last node points to the first node, enabling continuous cyclic traversal without reaching null, which perfectly models the circular behavior of CPU time allocation.
- This circular structure avoids special cases for the end of the list, making process traversal seamless and efficient.

3. Doubly Linked List (DLL)

Application: Implementation of a **Browser's Back and Forward Navigation**.

Justification:

- Browsers need to allow navigating both backward and forward through a history stack.
- DLL allows bi-directional traversal with pointers to both the previous and next nodes.
- The ability to efficiently move back and forth with O(1) complexity in either direction makes DLL ideal here.
- Singly linked lists, by contrast, cannot traverse backward without extra memory or traversal from the head.

4. Circularly Doubly Linked List (CDLL)

Application: Music Playlist or Media Player Track Navigation.

Justification:

- Music playlists typically loop continuously; after the last song is played, the player returns to the first song automatically.
- Users often navigate forward and backward through tracks arbitrarily.
- CDLL supports bi-directional traversal (like DLL) and circular looping (like CSLL).
- This structure intuitively models the playlist behavior with no null pointers, allowing smooth forward/backward and repeat-play navigation.

Summary Table

Linked List Type	Application	Justification Summary
Singly Linked List (SLL)	Stack	Efficient push/pop at one end, no backward traversal needed
Circularly Singly Linked List (CSLL)	Round Robin Scheduling	Circular traversal without null, models process rotation
Doubly Linked List (DLL)	Browser Back/Forward Navigation	Supports fast bi-directional traversal

Circularly Doubly Linked List (CDLL)	Music Playlist Navigation	Supports circular and bi-directional navigation seamlessly

Conclusion

Each linked list type offers unique traversal capabilities and structural properties that make it best suited for specific applications. Selecting the right linked list for a problem optimizes the efficiency and simplicity of the solution.

Q2: We can test whether a node ‘ m’ is a proper ancestor of a node ‘ n’ by testing whether ‘ m’ precedes ‘ n’ in X-order but follows ‘ n’ in Y-order , where X and Y are chosen from {pre, post, in}. Determine all those pairs X and Y for which this statement holds. (20 Marks)

Solution: **Problem Restatement:**

We want to determine for which pairs of traversal orders X and Y, chosen from {pre, in, post} traversal orders of a binary tree, the following condition holds:

> Node m is a **proper ancestor** of node n if and only if
> m **precedes** n in X-order traversal but
> m **follows** n in Y-order traversal.

Background Concepts

- **Binary Tree Traversals:**
  - **Preorder (pre):** Visit node first, then left subtree, then right subtree.
  - **Inorder (in):** Visit left subtree, then node, then right subtree.
  - **Postorder (post):** Visit left subtree, then right subtree, then node.
- **Proper Ancestor:** Node m is a proper ancestor of node n if m lies on the path from the root to n, and m ≠ n.
- The statement tests relative **positions** of m and n in two traversal sequences.

Step 1: Understand the condition

For node m to be an ancestor of n, the ordering condition in traversals must reflect the ancestor-descendant hierarchy.

We must find all pairs (X, Y) such that:

m comes before n in the X-order traversal, and m comes after n in the Y-order traversal,

if and only if m is a proper ancestor of n.

Step 2: Analyze known traversal orderings

Let's analyze the relative orderings of ancestor and descendant nodes in different traversals.

- **Preorder (pre):** Ancestors always appear **before** their descendants.
- **Inorder (in):** Ancestors can appear **before or after** descendants depending on the tree shape (left-heavy or right-heavy).
- **Postorder (post):** Ancestors always appear **after** their descendants.

Step 3: Test all pairs

There are 9 pairs possible from {pre, in, post}:

Pair (X, Y)	Interpretation	Does condition hold?	Reason
(pre, pre)	m before n in pre and after n in pre?	**No**	Can't be both before and after in same order.
(pre, in)	m before n in pre, m after n in in?	**Yes**	Ancestor is before descendant in pre, after descendant in in-order.
(pre, post)	m before n in pre, m after n in post?	**Yes**	Ancestor before descendant in pre, after descendant in post.
(in, pre)	m before n in in, m after n in pre?	**No**	Preorder places ancestors before descendants.
(in, in)	m before n in in, m after n in in?	**No**	Can't be before and after in same traversal.
(in, post)	m before n in in, m after n in post?	**No**	In-order does not guarantee ancestor always before descendant.
(post, pre)	m before n in post, m after n in pre?	**No**	Postorder puts ancestor after descendant, pre puts ancestor before descendant; no match.
(post, in)	m before n in post, m after n in in?	**No**	m before n in postorder contradicts ancestor relationship.
(post, post)	m before n in post, m after n in post?	**No**	Can't be both before and after in same order.

Step 4: Summary of Valid Pairs

The valid pairs (X, Y) for which the statement holds are:

- (pre, in)
- (pre, post)

Step 5: Explanation for valid pairs

- **(pre, in):**
  In **preorder**, ancestors appear before descendants, so m precedes n. In **inorder**, an ancestor m appears after n if n is in the left subtree of m. Hence, the condition holds generally.

- **(pre, post):**
  In **preorder**, ancestors appear before descendants. In **postorder**, ancestors are visited after all descendants, so ancestor m follows descendant n. This matches the condition perfectly.

Final Answer

Pair (X, Y)	Condition Holds?	Reason
(pre, in)	Yes	Ancestor before descendant in preorder; after in inorder.
(pre, post)	Yes	Ancestor before descendant in preorder; after in postorder.
All others	No	Ordering contradictory or impossible for ancestor-descendant relations.

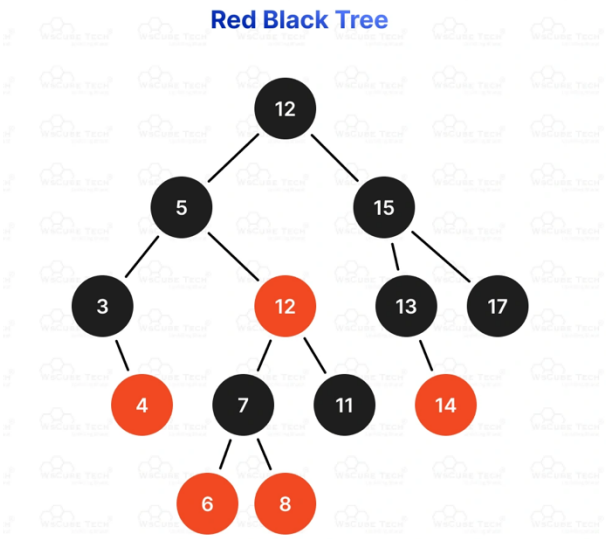
Q3: Explain Left Leaning Red Black Trees. What are their advantages and disadvantages? (20 Marks)

Solution: Left-Leaning Red-Black Trees: Explanation, Advantages, and Disadvantages

Left-Leaning Red-Black Trees (LLRBTs) are a variant of Red-Black Trees (RBTs), which are self-balancing binary search trees designed to maintain balanced height for efficient operations. LLRBTs simplify the implementation of standard RBTs by enforcing a "left-leaning" property, reducing the complexity of maintaining balance.

Explanation of Left-Leaning Red-Black Trees

Background: Red-Black Trees



A Red-Black Tree is a binary search tree with the following properties to ensure balance:

1. **Node Color**: Each node is either red or black.
2. **Root Property**: The root is black.
3. **Leaf Property**: All leaves (NIL nodes) are black.
4. **Red Property**: A red node cannot have a red child (no two red nodes are adjacent).
5. **Black-Height Property**: For any node, all paths from that node to its descendant leaves have the same number of black nodes.

These properties ensure the tree's height is at most (2 \log(n + 1)), guaranteeing (O(\log n)) time for search, insertion, and deletion.

Left-Leaning Red-Black Trees

LLRBTs simplify RBTs by adding a constraint: all red nodes must be left children (i.e., a node's left child can be red, but the right child cannot). This "left-leaning" property eliminates certain cases that arise in standard RBTs, simplifying the balancing logic. LLRBTs maintain the core RBT properties but reframe them to align with the left-leaning constraint.

Key Properties of LLRBTs

1. **Node Color**: Nodes are red or black.
2. **Root Property**: The root is black.
3. **Leaf Property**: NIL nodes (leaves) are black.

4. **Red Property**: No two red nodes can be adjacent, and red nodes are always left children.
5. **Black-Height Property**: All paths from a node to its leaves have the same number of black nodes.
6. **Left-Leaning Constraint**: If a node has a red child, it must be the left child. A right red child is not allowed.

The left-leaning property ensures that red links (representing parent-child relationships where the child is red) are oriented to the left, reducing the number of configurations to consider during insertions and deletions.

Correspondence to 2-3 Trees

LLRBTs are designed to correspond directly to 2-3 trees (a type of B-tree where nodes can have 2 or 3 children):

- A **black node** with two black children in an LLRBT corresponds to a single node in a 2-3 tree.
- A **black node** with a red left child corresponds to a 2-node in a 2-3 tree, where the red child represents a temporary "glue" to form a 3-node.
- The left-leaning property ensures that 3-nodes in the 2-3 tree are represented consistently (the middle key of a 3-node becomes the black parent, and the smaller key becomes the red left child).

This correspondence simplifies the balancing operations, as LLRBTs mimic the splitting and merging of 2-3 trees but use a binary tree structure.

Operations in LLRBTs

LLRBTs support standard binary search tree operations (search, insert, delete) with modifications to maintain the left-leaning and RBT properties.

1. Search

- Search is identical to a standard binary search tree, as colors and the left-leaning property do not affect the key comparison process.
- Time Complexity: (O(\log n)), as the tree is balanced.

2. Insertion

Insertion in LLRBTs follows these steps:

1. **Standard BST Insertion**: Insert the new node as in a binary search tree, coloring it red to minimize disruption to black-height.
2. **Fix Violations**: After insertion, the tree may violate LLRBT properties (e.g., two red nodes adjacent, right red child, or unbalanced black-height). Fix using:

- **Left Rotation**: If a right child is red, rotate left to make the left child red, maintaining the left-leaning property.
- **Right Rotation**: If two left red nodes are adjacent (parent and child both red), rotate right and recolor to resolve the violation.
- **Color Flip**: If a node has two red children, flip colors (parent becomes red, children become black) to maintain black-height, akin to splitting a 3-node in a 2-3 tree.

3. **Root Recoloring**: Ensure the root is black after fixing.

Example:

- Insert a node into a tree with nodes A (black), B (red, left child of A).
- If the new node creates a right red child, perform a left rotation.
- If two red nodes are adjacent on the left, perform a right rotation and recolor.
- Time Complexity: (O(\log n)), as rotations and color flips are constant-time, and the tree height is logarithmic.

3. Deletion

Deletion is more complex but simplified in LLRBTs compared to standard RBTs:

1. **Standard BST Deletion**: Find the node to delete and replace it with its successor (or predecessor), as in a BST.
2. **Fix Violations**: Deletion may disrupt black-height or create red-red violations. Use rotations and color flips to restore properties, ensuring left-leaning red links.
3. **Maintain Left-Leaning Property**: Adjust right red links via rotations.

- Time Complexity: (O(\log n)), though deletion involves more cases than insertion.

Advantages of LLRBTs

LLRBTs offer several benefits over standard RBTs and other balanced trees:

1. **Simplified Implementation**:
   - The left-leaning constraint reduces the number of cases to handle during insertion and deletion. Standard RBTs require handling both left and right red children, doubling the complexity of rotations and recoloring.
   - Only three main operations (left rotation, right rotation, color flip) are needed for insertion, making the code more concise and easier to verify.
2. **Fewer Rotations**:
   - By enforcing left-leaning red links, LLRBTs minimize the number of rotations required during balancing, as right red links are immediately corrected.
3. **Direct Correspondence to 2-3 Trees**:

- The mapping to 2-3 trees provides an intuitive framework for understanding balancing operations, as LLRBTs mimic 2-3 tree splits and merges in a binary structure.

4. **Comparable Performance**:
   - Like standard RBTs, LLRBTs guarantee (O(\log n)) time for search, insertion, and deletion due to the balanced height (at most (2 \log(n + 1))).
   - The left-leaning property does not degrade performance compared to standard RBTs.
5. **Ease of Debugging and Maintenance**:
   - The reduced case complexity makes LLRBTs easier to debug and maintain, especially for developers implementing balanced trees from scratch.

Disadvantages of LLRBTs

While LLRBTs simplify implementation, they have some drawbacks:

1. **Slightly Higher Constant Factors**:
   - The left-leaning constraint may require additional rotations to enforce (e.g., converting right red links to left red links), potentially increasing the constant factor in operations compared to standard RBTs in some cases.
   - However, this overhead is minimal and often offset by simpler code.
2. **Less Flexibility**:
   - The strict left-leaning property reduces flexibility in tree configurations. Standard RBTs allow red links on either side, potentially optimizing certain workloads where right-leaning structures are natural.
3. **Complex Deletion**:
   - While simpler than standard RBTs, deletion in LLRBTs remains more complex than insertion. Maintaining the left-leaning property and black-height during deletion requires careful handling of multiple cases, which can still be error-prone.
4. **No Significant Performance Advantage**:
   - LLRBTs do not offer asymptotic performance improvements over standard RBTs or other balanced trees like AVL trees. The primary benefit is implementation simplicity, not runtime efficiency.
5. **Limited Adoption**:
   - LLRBTs are less widely used than standard RBTs or AVL trees in standard libraries (e.g., Java's TreeMap uses standard RBTs). This may limit community support and resources for implementation.

Comparison to Other Balanced Trees

- **Vs. Standard RBTs**: LLRBTs are easier to implement due to fewer cases but may incur slight overhead from enforcing left-leaning links. Both have (O(\log n)) performance.

- **Vs. AVL Trees**: AVL trees are stricter in balancing (maintaining height difference ≤ 1), potentially leading to more rotations but slightly better height balance. LLRBTs are simpler to code and sufficient for most applications.
- **Vs. B-Trees**: B-trees (like 2-3 trees) are more suitable for disk-based storage due to higher branching factors, while LLRBTs are optimized for in-memory binary trees.

Applications

LLRBTs are suitable for applications requiring balanced binary search trees with simpler implementation, such as:

- **Symbol Tables**: In compilers or interpreters for efficient key-value storage.
- **Databases**: For in-memory indexing where balanced trees are needed.
- **Priority Queues**: With modifications to support priority-based operations.
- **Educational Purposes**: Teaching balanced trees due to their intuitive 2-3 tree correspondence.

Conclusion

Left-Leaning Red-Black Trees are a simplified variant of Red-Black Trees that enforce a left-leaning property for red nodes, reducing the complexity of balancing operations. Their primary advantage is implementation simplicity, with fewer cases to handle during insertion and deletion, making them ideal for educational purposes and scenarios where code maintainability is critical. However, they may introduce slight overhead due to the left-leaning constraint and offer no asymptotic performance improvement over standard RBTs.

Q4: Write a short note on the recent developments in the area of finding minimum cost spanning trees. (20 Marks)

Solution: Recent Developments in Minimum Cost Spanning Trees

Introduction

The **Minimum Cost Spanning Tree (MST)** problem is a cornerstone of graph theory and combinatorial optimization. It involves finding a spanning tree connecting all vertices in a weighted, undirected graph such that the total edge weight is minimized. While foundational algorithms like Kruskal's and Prim's have been well-studied, ongoing research continues to expand the capabilities and efficiency of MST solutions, particularly under new computational models, dynamic settings, and complex real-world constraints.

1. Dynamic and Incremental MST Algorithms

Dynamic MSTs

Recent years have witnessed significant progress in algorithms that efficiently update the MST when a graph undergoes changes – such as edge insertions, deletions, or weight updates – instead of recomputing from scratch.

- **Batch-Incremental Algorithms:** State-of-the-art methods efficiently handle batches of edge insertions or deletions. Some parallel algorithms can now update MSTs for large, sparse graphs in sublinear time per update, employing data structures that maintain compressed path information and enable quick rebalancing.
- **Dynamic Gallager-Humblet-Spira (DGHS) Algorithm:** Designed for wireless sensor and ad hoc networks, DGHS maintains MSTs in environments with constantly changing topology by exchanging only local information, thus adapting to network dynamics while minimizing communication overhead.
- **Temporal Graphs:** New algorithms handle graphs whose structure evolves over time, supporting network snapshots and maintaining MSTs across these changes for real-time applications.

Incremental Network Design

- Researchers have explored the **incremental network design problem with MST objectives**, minimizing the cost over a timeline as edges/nodes are added to the network. This variant introduces new challenges in computational complexity and optimization.

2. Distributed and Parallel MST Algorithms

Distributed MST Computation

As data sets grow massive and geographically distributed, building MSTs quickly across multiple machines or processors is a core challenge.

- **Classic GHS Algorithm:** The Gallager-Humblet-Spira algorithm remains a landmark distributed approach, and new variants further improve its efficiency in modern networking scenarios.
- **Near-Optimal Performance:** Advanced distributed techniques can now construct MSTs in nearly optimal time with respect to the network's diameter and size. Recent innovation targets resource-aware execution, minimizing the number of active ("awake") rounds each node undertakes, which is vital for energy-constrained networks like IoT and wireless sensor systems.
- **Parallel Batch Updates:** Modern algorithms support parallel processing for large incoming edge batches, dramatically improving scalability in big-data and cloud-based environments.

3. Multi-objective and Specialized MST Variants

Multi-objective MSTs

- **Multi-criteria Optimization:** Recent research investigates MST problems with vector-valued edge costs, considering multiple objectives simultaneously (e.g., cost, reliability, latency). Dynamic programming approaches and improved pruning techniques have produced new methods for handling these complex cases.
- **Bi-objective and Constrained MSTs:** Work on MSTs that minimize not just cost but other criteria such as diameter or degree has grown, especially for transportation, telecommunications, and logistics planning.

Generalized MST Problems

- **Resource Allocation:** Algorithms now address MSTs under resource constraints, such as limited bandwidth or connectivity requirements, reflecting real network design needs.
- **Steiner and Bottleneck Trees:** New approximation and heuristic solutions for Steiner trees, which generalize MSTs by allowing certain nodes not to be included, have seen practical adoption.

4. Theoretical Advances

- **Provably Optimal Algorithms:** Researchers have constructed deterministic, comparison-based MST algorithms that reach known lower bounds for required comparisons, achieving theoretical optimality, even though practical runtime may not always be optimal for all input sizes.
- **Algorithmic Complexity:** Developments in algorithm analysis, particularly for dense or large graphs, refine our theoretical understanding of MST construction bottlenecks and guide implementation tuning.

5. Application in Emerging Fields

- **Bioinformatics:** Use of MSTs in evolutionary biology for clustering gene and protein interactions.
- **Machine Learning:** MST-based clustering, especially in unsupervised learning and image processing, benefits from scalable and dynamic MST algorithms.
- **Smart Networks:** The design of robust, energy-efficient sensor and communication networks leverages dynamic and distributed MST computation.

Summary Table: Major Areas of Recent Advancement

Area	Description	Example Applications
Dynamic & Incremental MST	Real-time updates to MST on changing graphs	Network management, streaming analytics
Distributed & Parallel MST	Construction across many machines/nodes	Big-data analytics, sensor networks

Multi-objective & Constrained	Multiple cost criteria, resource/bandwidth limits	Transport, communication, eco-routing
Theoretical Advances	Optimality proofs and complexity breakthroughs	Algorithm design, large-scale systems
Domain-specific Applications	ML, Bioinformatics, Smart infrastructure	Clustering tasks, biological networks

Conclusion

The last decade has radically advanced the state-of-the-art in minimum cost spanning trees. From dynamic and distributed algorithms, through robust multi-objective optimization, to theoretical breakthroughs, MST research now directly addresses the scalability, adaptability, and efficiency required by modern large-scale and real-time systems. These developments not only enhance classic algorithmic performance but also broaden the MST's applicability to new domains and complex, real-world challenges.